

FICHE DE TD 3/3 • DIFFICILE

Informatique • 3ème

Présentation du langage de définition des algorithmes

Informatique – classe de 3ème

Consigne : Les exercices suivants portent sur la structure formelle d'un algorithme (en-tête, déclarations, corps). Justifiez rigoureusement chaque réponse. La rédaction, la clarté du raisonnement et la précision du vocabulaire technique sont notées.

EXERCICE 1 Analyse structurelle et détection d'anomalies

On considère l'algorithme suivant, écrit dans un langage de définition d'algorithmes standard (type LDA), destiné à calculer le coût total d'une commande avec frais de port dégressifs.

```
Algorithme CalculCommande
Variables
    qte, prixU, remise : Entier
    totalHT, fraisPort, totalTTC : Reel
    clientFidele : Booleen
Constantes
    SEUIL_REMISE <- 100
    TAUX_REMISE <- 0.05
    PORT_GRATUIT <- 500
Debut
    Afficher("Quantité : "); Saisir(qte)
    Afficher("Prix unitaire : "); Saisir(prixU)
    Afficher("Client fidèle (Vrai/Faux) : "); Saisir(clientFidele)
    totalHT <- qte * prixU
    Si totalHT > SEUIL_REMISE Alors
        remise <- totalHT * TAUX_REMISE
    Sinon
        remise <- 0
    FinSi
    totalHT <- totalHT - remise
    Si (totalHT > PORT_GRATUIT) ET (clientFidele = Vrai) Alors
        fraisPort <- 0
    Sinon
        fraisPort <- 5.5
    FinSi
    totalTTC <- totalHT + fraisPort
    Afficher("Total à payer : ", totalTTC)
Fin
```

1. Identifiez et expliquez quatre erreurs ou incohérences majeures (typage, logique,

portée, syntaxe) présentes dans la section de déclaration ou dans le corps de l'algorithme. Pour chacune, proposez la correction exacte.

2. L'algorithme utilise une variable `remise` de type `Entier` pour stocker le résultat d'une multiplication par un réel (`TAUX_REMISE`). Analysez les conséquences sur l'exécution (troncature, arrondi) et proposez une réécriture rigoureuse de la zone de calcul de la remise sans changer le type de `remise`, en justifiant l'opérateur utilisé.
3. Réécrivez la structure conditionnelle de calcul des `fraisPort` en utilisant une structure `Selon` (ou `Cas`) imbriquée, en gérant explicitement les quatre combinaisons logiques possibles des deux conditions. Précisez l'avantage de cette écriture par rapport au `Si...Sinon initial`.

EXERCICE 2 Modélisation d'un système réel : Distributeur automatique

On souhaite concevoir la structure déclarative (uniquement l'en-tête et la zone de déclarations) d'un algorithme pilotant un distributeur de boissons chaudes. Le système gère : le stock de gobelets (entier), le niveau d'eau en mL (entier), le nombre de doses de café/thé/chocolat (trois entiers distincts), le prix de chaque boisson (trois réels constants), la pièce insérée par l'utilisateur (entier en centimes), le choix utilisateur (caractère : 'C', 'T', 'H'), et un drapeau de maintenance (booléen). Le distributeur rend la monnaie (entier).

1. Rédigez la section `Variables et Constantes` complète, en choisissant des identificateurs normalisés (camelCase ou snake_case), les types les plus appropriés (`Entier`, `Reel`, `Booleen`, `Caractere`, `Chaine`) et en affectant des valeurs par défaut réalistes aux constantes (prix, capacités maximales).
2. Justifiez le choix du type `Reel` pour les prix plutôt que `Entier` (centimes), en discutant de la lisibilité vs la justesse arithmétique (virgule flottante).
3. Proposez une déclaration de type `Enregistrement` (ou `Structure`) nommée `StockBoisson` regroupant les doses et le prix pour une boisson. Réécrivez alors la déclaration des trois boissons sous forme d'un `Tableau` de `StockBoisson`. Expliquez en quoi cette structure facilite la maintenance du code (ajout d'une boisson).

EXERCICE 3 Preuve de correction partielle et invariants

Soit l'algorithme suivant calculant le PGCD de deux entiers strictement positifs `a` et `b` par la méthode des soustractions successives (version non optimisée d'Euclide).

```
Algorithme PGCD_Soustractions
Variables a, b, temp : Entier
Debut
    Saisir(a); Saisir(b)
```

```
TantQue a <> b Faire
  Si a > b Alors
    a <- a - b
  Sinon
    b <- b - a
  FinSi
FinTantQue
Afficher("PGCD = ", a)
Fin
```

1. Énoncez l'invariant de boucle $I(a, b)$ qui reste vrai à chaque itération du `TantQue`. Prouvez par récurrence sur le nombre d'itérations que cet invariant est conservé (initialisation, conservation).
2. Prouvez la terminaison de la boucle en exhibant un variant de boucle (fonction entière strictement positive et strictement décroissante). Justifiez pourquoi la condition $a <> b$ garantit la sortie.
3. En déduire la post-condition (résultat final) et prouver que l'algorithme retourne bien le PGCD initial.
4. Critiquez la structure de cet algorithme au regard de la complexité en temps (pire cas). Proposez une modification structurelle mineure (remplacement d'une instruction) pour passer en complexité logarithmique, sans changer la structure de contrôle `TantQue`.

EXERCICE 4 Synthèse : Conception d'un module de validation de notes

Un établissement scolaire souhaite un algorithme principal `Bulletin` qui appelle un sous-algorithme (procédure) `SaisirNote` pour valider la saisie d'une note sur 20. La procédure doit garantir : note réelle entre 0 et 20 inclus, coefficient entier strictement positif, et redemander la saisie en cas d'erreur avec message contextuel.

1. Écrivez l'en-tête complet de la procédure `SaisirNote` (nom, paramètres formels : mode entrée/sortie, types). Justifiez le choix du passage par référence (variable) pour la note et le coefficient.
2. Écrivez la structure déclarative locale de la procédure (variables locales nécessaires pour la boucle de contrôle).
3. Rédigez le corps de la procédure en utilisant une boucle `Repeter...JusquA` unique. La condition d'arrêt doit être une expression booléenne unique combinant les trois critères de validité.
4. Dans l'algorithme principal `Bulletin`, déclarez un Tableau de 10 notes et 10 coefficients. Écrivez la boucle d'appel à `SaisirNote` pour remplir ces tableaux. Ensuite, sans

écrire le code complet, décrivez précisément (pseudo-code commenté) la structure de calcul de la moyenne pondérée en gérant le cas où la somme des coefficients serait nulle (division par zéro).